

Prof. Dr. Armin Biere
Dr. Mathias Fleury
Florian Pollitt

Freiburg, 31/03/2025

Final Exam

Computer Architecture

Surname, Name: _____, _____

Matr. no.: _____

☐ CS/ ☐ ESE / Other:

☐ Master/ ☐ Bachelor/ Other:

Number of pages: 20

Available time: 90 Minuten

Allowed resources: None

Please check if you have received **all** sheets and fill in your name and your matriculation number on **all** sheets (also on the additional sheets you have received).

Question	Number of points	
	Possible	Aquired
1	16	
2	16	
3	16	
4	16	
5	16	
6	16	
Total	96	

Achieving **48** points is sufficient to pass the exam.

WITHDRAWING FROM AN EXAMINATION

In case of illness, you must supply proof of your illness by submitting a medical report to the Examinations Office. Please note that the medical examination must be done at the latest on the same day of the missed exam. In case of illness while writing the exam please contact the supervisory staff, inform them about your illness and immediately see your doctor. The medical certificate must be submitted latest 3 days after the medical examination. More information's: https://www.tf.uni-freiburg.de/studies/exams/withdrawing_exam.html

CHEATING/DISTURBING IN EXAMINATIONS

A student who disrupts the orderly proceedings of an examination will be excluded from the remainder of the exam by the respective examiners or invigilators. In such a case, the written exam of the student in question will be graded as 'nicht bestanden' (5.0, fail) on the grounds of cheating. In severe cases, the Board of Examiners will exclude the student from further examinations.

General information through this exam:

- Write your answers in Austrian, English, French, or German (alphabetical ordering). You can change between questions, but do not change within sentences.
- Whenever we ask for RISC-V code, we do not care about the exact instruction name. If you remember it, fine. If not, give it a descriptive acronym or name!
- Unless mentioned otherwise, we only consider 64-bit RISC-V.
- Unless specified otherwise, if we ask you to write programs, you do not need to save registers, nor remember which register has to be saved, nor remember how many registers there are.
- Every exercise gives the same number of points.
- If asked to consider the time or number of cycles to execute a program in a pipelined architecture, you do not need to consider the warm-up / close-down phase (i.e., you can consider that one instruction requires 1 cycle).
- This page, page 3, page 11, and page 20 have additional space for all solutions. If needed make sure to indicate **both**
 - on the additional page, which exercise your solution refers to;
 - in the corresponding exercise, that you used an additional page *with the page number*.

Failing to do so ends up in an exercise that is only partially corrected.

- The number of points per question is just an indication and might be corrected differently. But every exercise gives exactly 16 points.

Name: _____

Matriculation number: _____

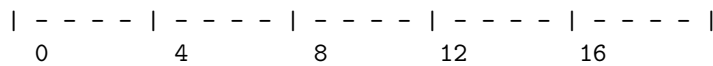
3

Additional pages for all exercises

Exercise 1 (2+5+5+4 points)

In this exercise, we are going to work on structures and how they are laid out in memory.

We distinguish between *aligned* and *unaligned* memory reads. A memory read is *aligned* when memory is read starting at a word boundary and unaligned if not. We consider that words have 4 bytes. Let's consider the following memory representation:

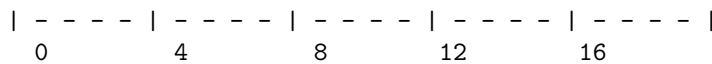


If you start reading 4 bytes at a multiple of 4 (0, 4, 8, ...), the reading is aligned. Otherwise it is not aligned. Reading unaligned memory is *slower* than reading aligned memory.

We consider the following C structure called **Aligned**

```
struct Aligned {
    int64_t a;
    int32_t b : 6; // only 6 bits
    int64_t c;
}
```

- a) Give the representation of **Aligned** in memory. Make sure that every member is aligned and that you use little-endian: if you use 3 bytes in a 4-byte region, you should use the 3 left-most ones and leave the last unused. The value of the unused bits is random.



- b) Complete the code below that multiplies two **Aligned** element-wise (**a*a** and **b*b**). You can overwrite padding (unused) bits, but cannot assume that they are zero.

```
ld      a4, __ (a1); load a1.b
ld      a5, __ (a0); load a0.b
ld      a2, __ (a1); load a1.a
ld      a3, __ (a0); load a0.a
andi    a4, a4, _____ ; what do we do here?
andi    a5, a5, _____ ; what do we do here?
mulw    a5, a5, a4
mul     a3, a3, a2
sw      a5, __ (a0); write a0.b
sd      a3, __ (a0); write a0.a
ret
```

Name: _____

Matriculation number: _____

5

- c) Our RISC-V processor has a RISC-V instruction `mulps` that can multiply 2 blocks of 4 consecutive `ints` together. In order to use them, they must be stored one after the other in memory, as one array of `ints`.

Write some pseudo-but-realistic RISC-V for such a multiplication using `mulps`.

- d) Can the ALU with SIMD be used for Booleans in order to calculate several conjunctions of Booleans at the same time? How much higher is the throughput?

Exercise 2 (6+2+4+4 points)

In this exercise, we study the branch prediction and we consider a RISC-V CPU with a branch predictor based on an old AMD processor (Athlon and Phenom CPUs).

Our CPU has a pipeline of *three* stages, called A, B, and C. You may assume that in the programs we are considering there are no data dependencies between instructions. **Branching instructions are resolved at the end of the last pipeline stage C, but they are predicted.** In the mean time the CPU continues executing instructions as predicted by a two-bit branch predictor.

Our CPU has a cache for instructions (L1i) with a capacity of only *five* instructions. The L1i cache contains the current instruction (given by the PC) and two surrounding instructions. More precisely, it contains the instructions at $\lfloor \frac{PC}{6} \rfloor$, $\lfloor \frac{PC}{6} \rfloor + 1$, ..., $\lfloor \frac{PC}{6} \rfloor + 5$. For example, when executing any of the instructions 2, 3, 4, or 5 the L1i cache contains the instructions 1, 2, 3, 4 and 5. **All five entries are populated at once at each step.**

The information to remember the status of the **two-bit** branch predictor is included in a separate register file. It remembers the status of all addresses.

- a) We consider the following program that sums up all values in an array if they are positive. Here are the given instruction addresses:

```

12:    ld a, 0(x)
13:    addi x, x, 4
14:    blti a, 0, 16; jump to the condition check, end of the loop
15:    add s, s, a
16:    blti x, 256, 12; less-than-immediate: jump to the beginning
17:    nope

```

Fill out what is happening to the L1i cache when executing the program on the next page:

- Execute the `blti` branch-less-than-immediate from line 14 (PC=14) instruction once, assuming it is taken. Stop when you PC=13 (the execution continues in question b)).
- the content the instruction cache (including addresses) during the execution will never change, but mark the current instruction with an arrow, as shown for the first step in the execution
- the branch prediction shows the state *before* the execution of the current pipeline stage
- the light gray lines have no semantics and can be ignored
- the number of empty steps templates given might be longer than necessary
- only add content in the branch predictor when it changes: as there is no branch with address 12, the status will never change and you do not have to write anything.

Name: _____

Matriculation number: _____

7

	branch predictor
14	strongly not taken
16	strongly taken

	L1i instructions		branch predictor
	12	15	14
	13	16	16
→	14	17	

A B C

	L1i instructions		branch predictor
	12	15	14
	13	16	16
	14	17	

A B C

	L1i instructions		branch predictor
	12	15	14
	13	16	16
	14	17	

A B C

	L1i instructions		branch predictor
	12	15	14
	13	16	16
	14	17	

A B C

	L1i instructions		branch predictor
	12	15	14
	13	16	16
	14	17	

A B C

	L1i instructions		branch predictor
	12	15	14
	13	16	16
	14	17	

A B C

continues on the next page in the next question!

b) Repeated program for easier follow-up

```

12:    ld a, 0(x)
13:    addi x, x, 4
14:    blti a, 0, 16; jump to the end of the loop
15:    add s, s, a
16:    blti x, 256, 12; jump to beginning
17:    nope

```

Now continue executing the program starting with instruction 14 assuming that it is not taken. Do not forget to copy the branching heuristic! Stop when 15 is fully executed and you have updated the branch predictor.

L1i instructions	branch predictor	A B C state following last from a)			
12 15	14				
13 16	16 strongly take				
→ 14 17					

L1i instructions	branch predictor	A B C			
12 15	14				
13 16	16				
14 17					

L1i instructions	branch predictor	A B C			
12 15	14				
13 16	16				
14 17					

L1i instructions	branch predictor	A B C			
12 15	14				
13 16	16				
14 17					

c) Now we consider a branchless version of the program that uses a conditional move (`cmov a, b, c`: if `a >= 0` is true, then write `b` to `a` else `c` to `a`). The idea is that we always sum, but we sum with '0' if the content of the memory is not positive. The `cmov` has one drawback: it always introduces one extra cycle, represented by the `nope` here.

```

12:    ld a, 0(x)
13:    addi x, x, 4
14:    nope; cmove takes one more cycle, so adding a nope
15:    cmov a, a, 0
16:    add s, s, a
17:    blti x, 256, 012

```

Execute the program again. Start with the `nope` and stop when the PC is again on the `nope`.

Name: _____

Matriculation number: _____

9

	branch predictor
17	strongly take

	L1i instructions		branch predictor
	12	15	14
	13	16	16
→	14	17	

A B C

	L1i instructions		branch predictor
	12	15	17
	13	16	
	14	17	

A B C

	L1i instructions		branch predictor
	12	15	17
	13	16	
	14	17	

A B C

	L1i instructions		branch predictor
	12	15	17
	13	16	
	14	17	

A B C

	L1i instructions		branch predictor
	12	15	17
	13	16	
	14	17	

A B C

	L1i instructions		branch predictor
	12	15	17
	13	16	
	14	17	

A B C

	L1i instructions		branch predictor
	12	15	17
	13	16	
	14	17	

A B C

- d) Based on the questions a) (incorrectly predicted branch), b) (correctly predicted branch), and c) (branchless programming), explain which of the two programs is better when the branch prediction works great and which one is better when the branch prediction is always wrong. Give the number of cycles!

Name: _____

Matriculation number: _____

11

Additional pages for all exercises

Exercise 3 (3+4+2+1+6 points)

For this exercise, we will consider a square-root like function and we will verify it. We consider unsigned two-bit numbers.

We consider a circuit such that: $\alpha + 1 = \beta^2 + r$ where α is the input and β (square root) and r (the rest) are the output. For example for $\alpha = 11_2 = 3$, we have $\beta = 10_2 = 2$ and $r = 0$.

The inputs are $\alpha = [a_2 a_1]_2$ and $\beta = [b_2 b_1]_2$ and $r = [r_2 r_1]_2$.

- a) Write the correctness equations over $\mathbb{Z}$.
- b) We consider a circuit calculating β and r from the inputs:

$$b_1 = \neg(a_2 \wedge a_1)$$

$$b_2 = (a_1 \wedge a_2)$$

$$r_2 = \neg a_1 \wedge a_2$$

$$r_1 = a_1 \wedge \neg a_2$$

Translate all the equations as polynomials.

- c) We also know that $b_1 = \neg a_1 \vee \neg a_2$. Use the polynomial representation to prove that the equation you obtained in b is the same as the translation of $\vee$.
- d) Prove that $b_1 b_2 = 0$
- e) Now do the first three meaningful steps of the calculation.

Hint: the full (very detailed) calculation takes 5 steps, but we only ask you for the first three to make the correction easier, but you might prove correctness in your 3 steps.

Your solution to exercise 3:

Name: _____

Matriculation number: _____

13

Additional pages for exercise 3 – page 3

Exercise 4 (2+8+6 points)

We use the usual five pipeline stages – IF, ID, EX, MEM, and WB – and the optimization with determining branch conditions for branches with a separate comparator/adder during the ID phase.

Assume that there is no branch prediction in this exercise, so the CPU waits for the branch destination to be found before executing the next instruction.

Throughout this exercise we further assume that the **ble** branch is **not taken**.

- a) Add arrows to represent all read after write dependencies between registers:

	add	a5,a0,zero
	slli	a1,a1,2
	add	a3,a5,a1
	addi	a0,zero,0
.L6:	lw	a4,0(a0)
	ble	a4,a2,.L5 // not taken
	addw	a0,a0,a4
	addi	a5,a6,4
.L5:	bne	a3,a5,.L6

- b) Indicate in the same program below what kind of forwarding happens and from which pipeline stage (IF, ID, EX, MEM, WB) to which pipeline stage this happens:

```
        add    a5,a0,zero
        slli   a1,a1,2
        add    a3,a5,a1
        addi   a0,zero,0
.L6:     lw     a4,0(a0)
        ble    a4,a2,.L5 // not taken
        addw   a0,a0,a4
        addi   a5,a6,4
.L5:     bne    a3,a5,.L6
```

- c) Indicate in the same program where stalls happens and what kind of hazard it is:

```
        add    a5,a0,zero
        slli   a1,a1,2
        add    a3,a5,a1
        addi   a0,zero,0
.L6:     lw     a4,0(a0)
        ble    a4,a2,.L5 // not taken
        addw   a0,a0,a4
        addi   a5,a6,4
.L5:     bne    a3,a5,.L6
```

Exercise 5 (5+3+3+2+1+1+1 points)

Through this exercise indicates that you have to write a number, ☐ if you need to tick with ✓ (one possible answer per line), and ☐ if you need to tick with either ✓ (yes) or ✗ (no) or nothing (pass) for each.

a) (Grading: -0.25 for wrong/0 skipped/0.25 for correct)

Fill in the table using ✓ when you need more of the column when adding a feature and ✗ if not.

	Bigger register files	More register files	Bigger ALU	More ALUs	
SIMD	☐	☐	☐	☐	(each: ✓/✗)
SMT/hyperthreading	☐	☐	☐	☐	(each: ✓/✗)
Multiple cores	☐	☐	☐	☐	(each: ✓/✗)
Multiple issues	☐	☐	☐	☐	(each: ✓/✗)
Graphics card	☐	☐	☐	☐	(each: ✓/✗)

b) To represent disjunction (OR) in an AIG in the simplest possible way, you need (put numbers):

conjunctions (ANDs) and negations. Draw the corresponding AIG.

c) (Grading: 1 for correct, no point for multiple ✓ per row)

Using only ✓, fill the following table indicating *the most important* factor for speed to evaluate an AIG given an assignment to the input variables to true or false:

AIG evaluator \ Factor	Span (longest path)	Amount of work	Number of nodes	
Hardware	☐	☐	☐	(one ✓ per row!)
Single-threaded software	☐	☐	☐	(one ✓ per row!)
Multiple-threaded software	☐	☐	☐	(one ✓ per row!)

- d) We consider two CPU cores that share one component, the floating point unit (FPU), to save space on the die. The FPU component executes each instruction in one cycle.

We consider the following sharing policy: core 0 is used for even clock cycles (0, 2, 4, ...) and core 1 for odd clock cycles (1, 3, 5, ...).

Assuming an infinite program that uses the component for each instruction, what is the CPI?

Hint: the infinite part means that we can ignore initial waiting time for core 1.

.....

- e) We consider two CPU cores that share one component, the floating point unit (FPU), to save space on the die. The FPU component executes each instruction in one cycle.

We consider the following sharing policy: we use the *newest* instruction that came in and in case of ties we use the instruction for core 0.

Assuming that each core is using the component and that we have an infinite program. What is the CPI for core 1? Tick the correct answer with ✓!

Hint: the infinite part means that we can ignore initial waiting time for core 1

☐ 0 ☐ 1 ☐ $+\infty$ ☐ 2 (one ✓ only)

- f) In the setup of question e, what is the CPI of core 2?

☐ 0 ☐ 1 ☐ $+\infty$ ☐ 2 (one ✓ only)

- g) Is the behavior of question f expected? In one sentence, explain or explain how to fix it.

Exercise 6 (6+6+2+2 points)

In this exercise we consider the two different eviction policies: LRU (least-recently used) and PLRU (pseudo-LRU).

We consider a cache of associativity 4 containing a *single* cache set. We will also ignore the valid flag and assume the data is always valid.

Hint: In the initial cache, there is an unused entry ℓ . The letter stands for both LRU *looser* and PLRU *lucky* loser.

a) We consider the following read pattern

```
read t
read e
read t
read c
read t
read d
```

LRU follows by evicting the oldest unused entry in the cache. Therefore, you track the age (here a number from 1 to 4, 1 = newest, 4 = oldest) and replace the oldest one if you need to add a new entry. When using/updating an entry, you mark as newest and renumber the others, keeping the relative order.

Fill the status of the cache. List the number of cache misses at each time stamp. If the content does not change, you do not have to repeat it.

Timestamp	read t	read e	read t	read c	read t	read d	
Age (1 to 4)	1 4 2 3						
Content	t ℓ c d						
Cache misses:	0	+	+	+	+	+	=
Cache hits:	0	+	+	+	+	+	=

- b) Now we use Pseudo-LRU (LRU). It is an approximation of LRU, by using two levels of LRU. The cache set is divided into sides, the left side A and the right side B. Then each side is divided into two cells where we use LRU again. To track the two LRU information, we have two bits to track: ✓ for the most-recently used side. Then in each side: the most-recently used entry also gets a ✓. When using/updating the cache, you only have to update *one* side (the most-recently used entry of the other side does not change). The evicted entry is always in the “oldest side” (no ✓) , the “oldest” entry (no ✓).

Fill the status of the cache with the same read pattern from a:

Timestamp	read t		read e		read t		read c		read t		read d	
Side	A	B	A	B	A	B	A	B	A	B	A	B
LRU of the side (✓)	✓											
LRU within the side (✓)	✓	✓										
Content	t	ℓ	c	d								
Cache misses:	0	+		+		+		+		+		=
Cache hits:	0	+		+		+		+		+		=

- c) Given the number of misses and hits, which scheme is better for this example? Explain why PLRU does not work in this example!
- d) LRU often performs better. Why doesn't Intel use it for its 16-way associative caches?

Additional pages for all exercises